

Computer Networks
CL3001

LAB – 07 NOTES
NS3

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES,
KARACHI CAMPUS
FAST SCHOOL OF COMPUTING (AI & DS, CS, CY, SE)
SPRING 2025

Computer Networks Lab 07

Course: Computer Networks (CL3001)

Semester: Spring 2025

Instructor: Muhammad Nadeem Ghouri

Note:

- Maintain discipline during the lab.
 - Listen and follow the instructions as they are given.
 - Just raise hand if you have any problem.
 - Completing all tasks of each lab is compulsory.
 - Get your lab checked at the end of the session.
-

Lab Objective

- Introduction to NS3 network simulator.
- Simple network simulation in NS3.
- Implementation of TCP congestion control & simulation in NS3.

NS3

1. Introduction

NS3 (Network Simulator 3) is a **discrete-event simulator** used for **networking research**. It allows for the simulation of network protocols such as **TCP, UDP, and various routing mechanisms** over **wired** and **wireless** networks. NS3 is primarily used in **UNIX-based environments**.

1.Languages Used: NS3 primarily uses **C++** for simulations, but Python can also be used.

2.Scripting: Unlike NS2, which uses **Tcl**, NS3 scripts are typically written in **C++**.

3.Installation: NS3 must be installed on a UNIX-like system (Linux or macOS), and its dependencies should be installed beforehand.

NS3 is a discrete event simulator targeted at networking research. NS3 provides substantial support for simulation of TCP, UDP, routing & multicast protocols over wired & wireless (local & satellite) networks. This simulator is primarily UNIX based & the NS commands can either be entered via UNIX command prompt or by running a scripting language file, but scripting language is preferred way. NS3 uses Tcl (pronounced as 'tickle') as its scripting language, which is edited using a text editor & in this example we will use vi, gedit or nano.

2. Introduction to vi

vi is a text editor that is available in all UNIX systems, other editors such as pico & emacs can also be used instead. Before you start for the first time, you must note that vi is a modal editor. A mode is like an environment. Different modes in vi interpret the same key differently. For example, if you're in insert mode, typing a adds an a to the text, whereas in command mode, typing a puts you in insert mode because a is the key abbreviation for the append command. If you get confused about what mode you're in, press the Escape key. Pressing the Escape key always returns you to command mode & if you are already in command mode, it simply beeps to remind you of the fact.

Since NS3 scripts may require editing in a UNIX environment, the lab introduces the **vi-text editor**:

- **Modal Editor:** Operates in two main modes:
 - **Command Mode:** For **managing the document** (e.g., saving, quitting). Press Esc to enter this mode.
 - **Insert Mode:** For **typing or editing text**. Enter this mode by pressing a (append).
- **Basic Commands:**
 - **Start vi:** vi (new file) or vi filename (existing file).
 - **Enter insert mode:** Press a.
 - **Save:** In command mode, type :w and press Enter.
 - **Quit:** :q (if no changes) or :q! (discard changes).
 - **Save and quit:** :wq.

When you are in command mode, you can manage your document; this includes the capability to change text, rearrange it, & delete it. Insert mode is when you maneuver the text area using the arrow keys.

For starting a new document, simply type vi after the command prompt to start the vi editor. The cursor will be located in the top left corner & each of the following lines will start with a tilde (~) denoting empty lines. Note that the vi editor is currently in command mode. In order to edit an existing file, type the name of the file along with the extension after typing vi & a space in the UNIX command prompt. To enter into insert mode simply press a on the keyboard. Then to save the edited document, type a colon (:) & the cursor will be located in the bottom left corner after the colon, followed by a w & then press enter. In order to quit vi editor type a colon followed by a q (:q), this will take you back to the UNIX prompt. You can also choose to quit editing without saving by typing :q!, & if you forget the exclamation mark (!) the system will issue a warning.

Download and Installation Guide for NS 3

<\\172.16.5.41\\SoftBox\\Open Source ISO Images\\Ubuntu 18\\Ubuntu1804.rar>

<https://www.nsnam.com/2025/01/ns3-installation-on-ubuntu-ns-343.html>

<https://www.nsnam.com/2020/04/download-vm-image-of-ns3-and-contiki-ng.html>

3. Setting Up NS3 in UNIX

To set up and run NS3, follow these steps:

Installation Steps (Ubuntu/Linux)

```
sudo apt update
sudo apt install -y g++ python3 cmake ninja-build
sudo apt install -y git mercurial python3-pip
sudo apt install -y qt5-qmake qtbase5-dev
sudo apt install -y gnuplot
git clone https://gitlab.com/nsnam/ns-3-dev.git
cd ns-3-dev
./ns3 configure --enable-examples --enable-tests
./ns3 build
```

Running NS3 Programs

1. Navigate to the NS3 directory:

```
cd ns-3-dev
```

2. Run a sample script:

```
./ns3 run scratch/first
```

4. Simple Simulation in NS3

First.cc

Explanation:

- **Purpose:** Simulates a **simple UDP echo client-server model over a point-to-point network**.
- **Key Steps:**
 1. **Node Creation:** Creates 2 nodes (NodeContainer nodes; nodes.Create(2);).
 2. **Link Setup:** Uses PointToPointHelper to connect nodes with a data rate of 5 Mbps and a 2 ms delay.
 3. **IP Assignment:** Assigns IP addresses in the range 10.1.1.0/24.
 4. **Applications:**
 - **UDP Echo Server:** Runs on node 1, listens on port 9, starts at 1s, stops at 10s.
 - **UDP Echo Client:** Runs on node 0, sends one 1024-byte packet at 2s with a 1s interval.
 5. **Simulation:** Runs and cleans up with Simulator::Run() and Simulator::Destroy().
- **Output:** Logs client/server activity (if enabled) to show packet transmission.

1. Node Creation

- The **NodeContainer** creates two nodes (`nodes.Create(2);`).
- These two nodes represent the client and the server.

2. Link Setup

- The **PointToPointHelper** is used to connect the nodes.
- Attributes set:
 - **Data Rate:** 5Mbps
 - **Delay:** 2ms
- The **Install** function attaches these configurations to the nodes.

3. Internet Protocol (IP) Assignment

- **InternetStackHelper** is installed on both nodes to enable networking capabilities.
- **Ipv4AddressHelper** assigns the **10.1.1.0/24** subnet.
- IP addresses are assigned to the nodes using the **Assign** function.

4. Application Setup

- **UDP Echo Server:**
 - Runs on **node 1**.
 - Listens on **port 9**.
 - Starts at **1s**, stops at **10s**.
- **UDP Echo Client:**
 - Runs on **node 0**.
 - Sends **one 1024-byte packet** to the server at **2s**.
 - Sends packets at **1s intervals**.

5. Simulation Execution

- **Simulator::Run()** executes the simulation.
- **Simulator::Destroy()** cleans up memory after execution.

Expected Output

If logging is enabled (**LogComponentEnable**), the simulation logs:

- Server startup and packet reception.
- Client sending the packet.
- Server echoing the packet back to the client.

```
#include "ns3/core-module.h"
#include "ns3/network-module.h"
#include "ns3/internet-module.h"
#include "ns3/point-to-point-module.h"
#include "ns3/applications-module.h"
using namespace ns3;

NS_LOG_COMPONENT_DEFINE ("FirstScriptExample");

int main (int argc, char *argv[])
{
    CommandLine cmd;
    cmd.Parse (argc, argv);
```

```

Time::SetResolution (Time::NS);
LogComponentEnable ("UdpEchoClientApplication", LOG_LEVEL_INFO);
LogComponentEnable ("UdpEchoServerApplication", LOG_LEVEL_INFO);

NodeContainer nodes;
nodes.Create (2);

PointToPointHelper pointToPoint;
pointToPoint.SetDeviceAttribute ("DataRate", StringValue
("5Mbps"));
pointToPoint.SetChannelAttribute ("Delay", StringValue ("2ms"));

NetDeviceContainer devices;
devices = pointToPoint.Install (nodes);

InternetStackHelper stack;
stack.Install (nodes);

Ipv4AddressHelper address;
address.SetBase ("10.1.1.0", "255.255.255.0");

Ipv4InterfaceContainer interfaces = address.Assign (devices);

UdpEchoServerHelper echoServer (9);
ApplicationContainer serverApps = echoServer.Install (nodes.Get
(1));
serverApps.Start (Seconds (1.0));
serverApps.Stop (Seconds (10.0));

UdpEchoClientHelper echoClient (interfaces.GetAddress (1), 9);
echoClient.SetAttribute ("MaxPackets", UintegerValue (1));
echoClient.SetAttribute ("Interval", TimeValue (Seconds (1.0)));
echoClient.SetAttribute ("PacketSize", UintegerValue (1024));

ApplicationContainer clientApps = echoClient.Install (nodes.Get
(0));
clientApps.Start (Seconds (2.0));
clientApps.Stop (Seconds (10.0));

Simulator::Run ();
Simulator::Destroy ();

return 0;
}

```


Second.cc

Explanation:

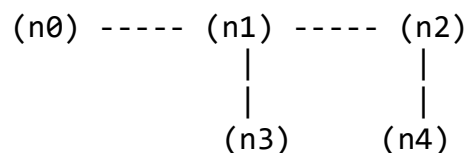
- **Purpose:** Simulates a hybrid topology with a **point-to-point link** and a **CSMA (LAN)** network.
- **Topology:**
 - n0 connects to n1 via point-to-point (10.1.1.0/24).
 - n1 connects to n2, n3, n4 via a CSMA LAN (10.1.2.0/24).
- **Key Steps:**
 1. **Node Creation:** 2 point-to-point nodes and 3 CSMA nodes (configurable via nCsm).
 2. **Link Setup:**
 - Point-to-point: 5 Mbps, 2 ms delay.
 - CSMA: 100 Mbps, 6560 ns delay.
 3. **IP Assignment:** Two subnets (10.1.1.0/24 for point-to-point, 10.1.2.0/24 for CSMA).
 4. **Applications:** UDP echo server on the last CSMA node, client on n0.
 5. **Extras:**
 - Enables packet capture (.pcap files) with EnablePcapAll.
 - Creates an animation file (second.xml) using AnimationInterface.
 6. **Routing:** Uses Ipv4GlobalRoutingHelper to populate routing tables.
- **Output:** Logs activity, generates .pcap files, and creates an animation file for visualization.

1. Purpose

- Simulates a simple hybrid network with a point-to-point link and a LAN.
- Demonstrates IP addressing, routing, and application-layer communication using UDP echo.
- Captures network activity via **.pcap** files and generates an animation file for visualization.

2. Network Topology

The script sets up the following network:



- **Point-to-Point Link** between **n0** and **n1** (Subnet: **10.1.1.0/24**).
- **CSMA LAN** connecting **n1**, **n2**, **n3**, and **n4** (Subnet: **10.1.2.0/24**).

3. Key Components

a. Node Creation

- **Point-to-Point Nodes:** `n0` and `n1`.
- **CSMA Nodes:** `n1`, `n2`, `n3`, `n4` (default is 3 extra CSMA nodes).

b. Link Configuration

- **Point-to-Point:**
 - Bandwidth: 5 Mbps
 - Delay: 2 ms
- **CSMA:**
 - Bandwidth: 100 Mbps
 - Delay: 6560 ns (~6.56 μ s)

c. IP Addressing

- `10.1.1.0/24` → Assigned to **point-to-point** link.
- `10.1.2.0/24` → Assigned to **CSMA LAN**.

d. UDP Echo Application

- **UDP Echo Server:** Runs on the last CSMA node (`n4`).
- **UDP Echo Client:** Installed on `n0`, sending one 1024-byte packet.

e. Packet Capture and Animation

- `EnablePcapAll()`: Captures network traffic in **.pcap** files.
- `AnimationInterface("second.xml")`: Creates a simulation animation file.

f. Routing

- `Ipv4GlobalRoutingHelper::PopulateRoutingTables()`: Ensures correct packet forwarding.

4. Expected Output

1. Simulation Logs:

- UDP Echo Client and Server communication logs.
- Routing table updates.

2. Captured Network Traffic (.pcap files):

- Contains packet exchange details for further analysis in Wireshark.

3. Animation File (second.xml):

- Visualizes network activity in NetAnim.

```
#include "ns3/core-module.h"
#include "ns3/network-module.h"
#include "ns3/csma-module.h"
#include "ns3/internet-module.h"
#include "ns3/point-to-point-module.h"
#include "ns3/applications-module.h"
#include "ns3/ipv4-global-routing-helper.h"
#include "ns3/netanim-module.h"
using namespace ns3;
```

```
NS_LOG_COMPONENT_DEFINE ("SecondScriptExample");
```



```

int main (int argc, char *argv[])
{
    bool verbose = true;
    uint32_t nCsma = 3;
    CommandLine cmd;
    cmd.AddValue ("nCsma", "Number of extra CSMA nodes/devices",
nCsma);
    cmd.Parse (argc, argv);
    if (verbose)
    {
        LogComponentEnable ("UdpEchoClientApplication",
LOG_LEVEL_INFO);
        LogComponentEnable ("UdpEchoServerApplication",
LOG_LEVEL_INFO);
    }

    nCsma = nCsma == 0 ? 1 : nCsma;
    NodeContainer p2pNodes;
    p2pNodes.Create (2);

    NodeContainer csmaNodes;
    csmaNodes.Add (p2pNodes.Get (1));
    csmaNodes.Create (nCsma);

    PointToPointHelper pointToPoint;
    pointToPoint.SetDeviceAttribute ("DataRate", StringValue
("5Mbps"));
    pointToPoint.SetChannelAttribute ("Delay", StringValue ("2ms"));
    NetDeviceContainer p2pDevices;
    p2pDevices = pointToPoint.Install (p2pNodes);

    CsmaHelper csma;
    csma.SetChannelAttribute ("DataRate", StringValue ("100Mbps"));
    csma.SetChannelAttribute ("Delay", TimeValue (NanoSeconds
(6560)));
    NetDeviceContainer csmaDevices;
    csmaDevices = csma.Install (csmaNodes);

    InternetStackHelper stack;
    stack.Install (p2pNodes.Get (0));
    stack.Install (csmaNodes);

    Ipv4AddressHelper address;
    address.SetBase ("10.1.1.0", "255.255.255.0");
    Ipv4InterfaceContainer p2pInterfaces;

```

```

p2pInterfaces = address.Assign (p2pDevices);

address.SetBase ("10.1.2.0", "255.255.255.0");
Ipv4InterfaceContainer csmaInterfaces;
csmaInterfaces = address.Assign (csmaDevices);

UdpEchoServerHelper echoServer (9);
ApplicationContainer serverApps = echoServer.Install
(csmaNodes.Get (nCsma));
serverApps.Start (Seconds (1.0));
serverApps.Stop (Seconds (10.0));

UdpEchoClientHelper echoClient (csmaInterfaces.GetAddress (nCsma),
9);
echoClient.SetAttribute ("MaxPackets", UintegerValue (1));
echoClient.SetAttribute ("Interval", TimeValue (Seconds (1.0)));
echoClient.SetAttribute ("PacketSize", UintegerValue (1024));

ApplicationContainer clientApps = echoClient.Install (p2pNodes.Get
(0));
clientApps.Start (Seconds (2.0));
clientApps.Stop (Seconds (10.0));

Ipv4GlobalRoutingHelper::PopulateRoutingTables ();

AnimationInterface anim ("second.xml");

Simulator::Run ();
Simulator::Destroy ();

return 0;
}

```

Code Explanation:

The following is the explanation of the script above. In general, an NS script starts with making a Simulator object instance.

- `set ns [new Simulator]`: generates an NS simulator object instance, and assigns it to variable ns (italics is used for variables and values in this section).

What this line does is the following: Initialize the packet format (ignore this for now).

- Create a scheduler (default is calendar scheduler).
- Select the default address format (ignore this for now)
- The "Simulator" object has member functions that do the following:
 - Create compound objects such as nodes and links (described later).
 - Connect network component objects created (ex. attach-agent).
 - Set network component parameters (mostly for compound objects).
 - Create connections between agents (ex. make connection between a "tcp" and "sink").
 - Specify NAM display options.

Most of member functions are for simulation setup (referred to as plumbing functions) and scheduling, however some of them are for the NAM display.

The "Simulator" object member function implementations are located in the "ns-2/tcl/lib/nslib.tcl" file.

TCP Congestion Control Simulation

In this example we will implement & simulate congestion control mechanism of TCP between two nodes. It consists of three parts which are as follows:

TCP congestion control, a mechanism to **manage network traffic and prevent overload**. The lab focuses on:

1. **Slow Start**: Exponential increase in packet sending rate until a threshold (ssthresh) is reached.
2. **Congestion Avoidance**: Linear (additive) increase in rate to probe network capacity.
3. **Congestion Detection**: Multiplicative decrease in rate when packet loss occurs.

TCP Variants:

- **TCP Tahoe**: Drops to 1 segment and restarts slow start after congestion.
- **TCP Reno**: Uses fast retransmit and recovery, halving the window instead of dropping to 1.

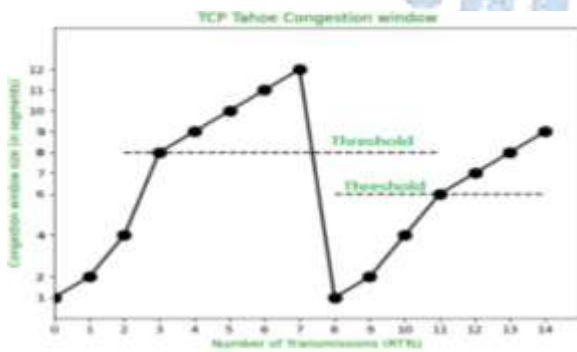


Figure 1 - TCP Tahoe

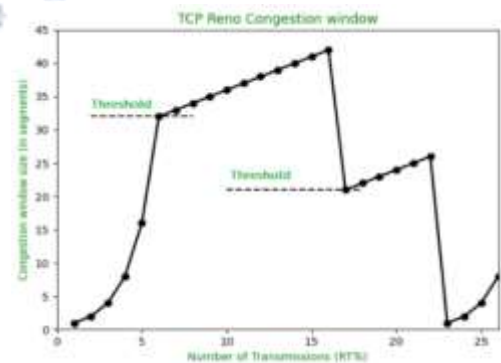


Figure 2 - TCP Reno

Lab Exercise

1. Apply TCP Congestion control mechanism in NS3 & plot the congestion window graph.

Helpful link for TCP congestion Task:

<https://www.youtube.com/watch?v=9rkN3FtOkaQ&t=885s>

